

Weather Station Build

This Weather Station is just an example of many possible builds using different materials. Depending on what sensors you choose to incorporate, this build is designed to collect up to 4 types of weather data: temperature, humidity, wind speed and (optionally) barometric pressure. Essentially, there are 4 major components that can be adapted to your design preference: Container (+fasteners), sensors, processor, and transmitter.

Components:

Container:

Purchased or Repurposed

Any container that is big enough to hold the components AND is waterproof (sun and rain). Take home food and soup containers.



<https://www.amazon.com/Containers-Disposable-Insulated-Microwaveable-Dishwasher/dp/B07BLTDD1V>



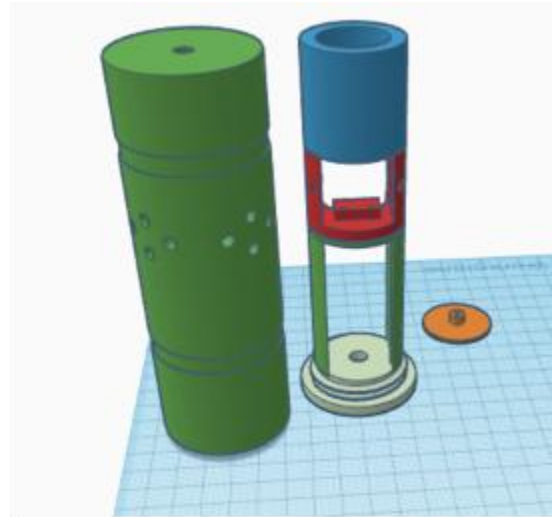
3D Printed Components.

You can design your own (or use our example seen below).



[https://github.com/CIJE-STEM/CIJE-](https://github.com/CIJE-STEM/CIJE-WeatherHub/tree/main/3D_files/CIJE_Weather_Station1)

[WeatherHub/tree/main/3D_files/CIJE_Weather_Station1](https://github.com/CIJE-STEM/CIJE-WeatherHub/tree/main/3D_files/CIJE_Weather_Station1)



That can be adapted via Tinkercad.

<https://www.tinkercad.com/things/kYxrZfITcn0-cije-weather-station?sharecode=hQLnVu-pN1FPCwDh6w1JOU05zTvHCK09-pb4Zir9fJ0>

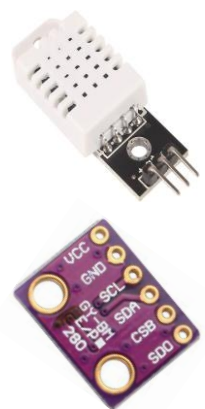
Fasteners:

The container needs to be anchored/attached to something stable to withstand wind and rain. Zip ties are the easiest to use.

Sensors:

Temperature/Humidity sensor

- **DHT22/DHT11** - I2C device measure temperature and humidity
- **BME280** - I2C device measures temperature, humidity, and barometric pressure.



Wind Speed:

Anemometer - We are suggesting an anemometer adapted from a DC motor wind turbine. The voltage output will need to be calibrated to the wind speed.

https://www.amazon.com/Turbine-Vertical-Turbines-Electricity-Generator/dp/B0BWWVQZTG/ref=sr_1_16

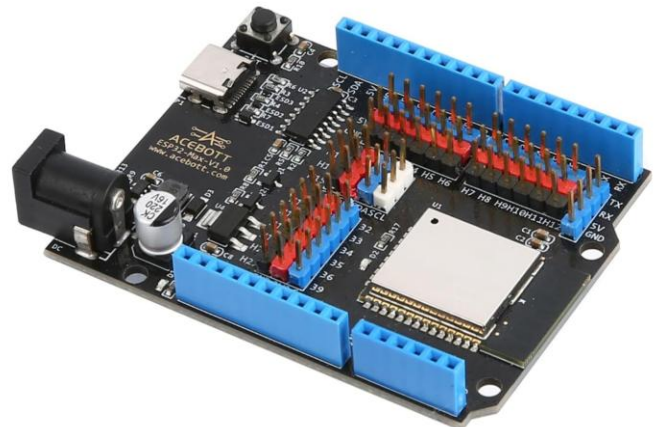
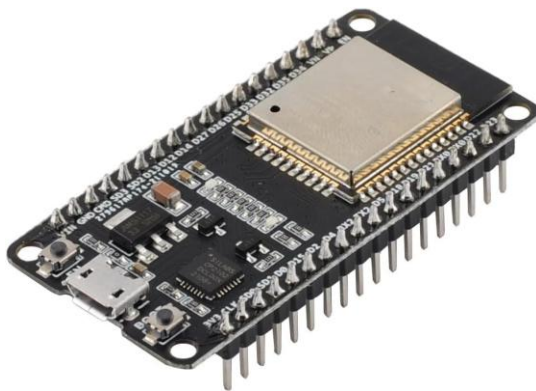
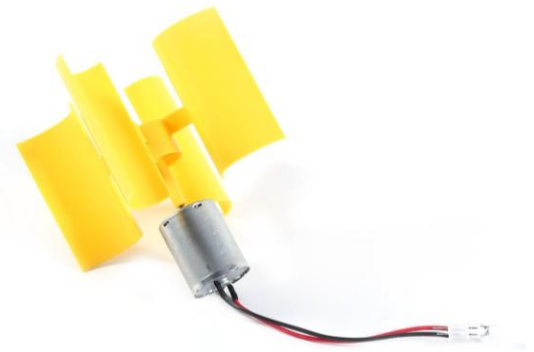
Time: (optional)

The **DS3231** A real time clock (RTC) is great to have to put the microprocessor to sleep and not use up the power (unless you manage to wire the power) when not in use / when not transmitting the data. There are other versions that are also available. This model has a rechargeable battery vs. a coin cell battery that will eventually need to be replaced.

https://www.amazon.com/Dorhea-DS3231-Module-Memory-Raspberry/dp/B08X4H3NBR/ref=sr_1_3?

Processor/Transmitter

ESP32 (Processor AND transmitter)



Build-Up Options:

Your weather station can be built with a wide variety of options which will be explored. These options may be individual or combined depending on your needs.

Here are possible configurations:

Basic:

Internal sensors require minimal fabrication.

- Temperature, Humidity
 - DHT-11, DHT22 Sensors.
 - AHT10 Sensor.
- Temperature, Humidity, Pressure.
 - BME280.
- Temperature, Humidity, Pressure, VOC's (Volatile Organic Compounds - Air Quality)
 - BME680.

Intermediate:

Sensors requiring some physical material fabrication, holes, mounting materials.

- (Basic Components - as listed above).
- Anemometer - DC Generator, resistors and a mounting bracket, hole for wiring.
- UV Sensor
 - Adafruit LTR-390 - requires a hole and weather protective window.

Advanced:

Sensors that require moderate to extensive mechanical fabrication and adaptation, external holes.

- Wind Direction Sensor - rotary magnetic, optical, or mechanical switch sensing with an air vane to detect air flow direction.
 - AS5600 - rotary magnetic position sensor.
 - A3144 Hall Effect sensor or A3144 Hall Effect sensor module board, magnet(s).
 - TCRT5000 Reflectance sensor(s), position disk.
- Participation Sensor (Rain Gauge)
 - A3144 Hall Sensor, 3D printed tipping bucket assembly.
 - LM393 Rain Drop Sensor Monitor.
 - Capacitive Analog Soil Sensor Module.
 - Rain gauge sensor with ultrasonic or TOF (Time of Flight) distance sensor.

The above options may be selected or combined to create your weather station. We will explore some of these options with design examples to help you get started.

First Steps:

- What is the intended application of your weather station?
 - Classroom project, school project?
 - Do you have an intended time frame for installation and use?
 - Can you get permission and a designated place to install the weather station?
- Consider the location of your weather station.
 - Is it to be inside or outside - Are measuring indoor air quality, lighting, etc.
 - Does the location have readily available power?
 - Does the location have readily available Wi-Fi?
 - Is the location easily accessible?
 - Is the location secure?
- What kind of data do you wish to collect?
 - Some sensors are easy to use and require very little maintenance others may require occasional repair or adjustment.
 - Are you considering microclimate measurement with multiple stations or having a relocatable station?
 - How will you share the data with your students, school, community?
 - Do you expect that your station and data will be used to help plan or make decisions based on the data you collect? For example, a community garden, air quality monitoring?

Based on the questions above hopefully you can formulate the intent and purpose of the weather station as well as what kind of data you wish to acquire/use.

Project Planning:

Consider the scope of your project and the resources that you will be able to apply. This project can be incremental, and scaled based on the resources that you have at hand. Consider a part of this project could be to build a case for expansion and permanence at your school, location. It may be useful to start small, demonstrate utility or interest and then expand.

Design Example - Basic - Web Based Reporting

In this example we want a weather station that can be located outside but next to a classroom window. We may find a better location later but for now we would like to have something that we can experiment with and easily observe. Power will be routed through the window using a 12V DC power adapter. Wi-Fi is available and reliable next to the window, and we have permission from our IT department to use the network. Our school is happy and interested in having the data posted on the CIJE Weather Station Hub site, so our IT department will allow outgoing internet traffic from our weather station.



Shown above is an image of a basic weather station using an ESP32 ([ACEBOTT](#)), a half bread board and a BME280 sensor. This is a very simple example of a web posting weather station.

We have some wiring to do, coding (most of it is provided) and some testing.

Weather Station Specifications:

#	Sensing Parameter	Sensor	Notes
1	Temperature	DHT22 - DPin - 3.3V-5V	Amazon - DHT22E280 not BMP280
	Humidity	DHT22 - DPin - 3.3V-5V	
#	Hardware	Type	
2	ESP32	ESP32 Board	ACEBOTT UNO format ESP32
3	Breadboard	Mini Breadboard	Makes for easy design changes
4	1N5817 Diode (2)	Overvoltage Protection	1N5817 Schottky Diode
5	10KΩ 1/4W Resistor	Anemometer V/Divider	10K resistor
6	1KΩ 1/4W Resistor	Anemometer V/Divider	1K resistor

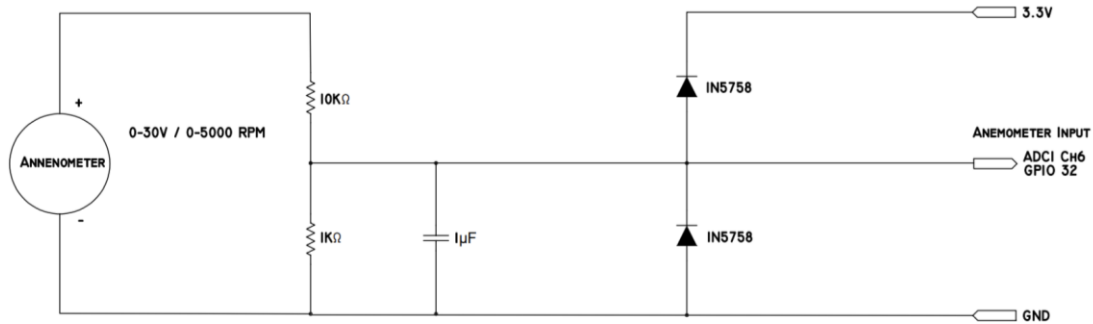
7	100K Ω 1/4W Resistor	Battery V/Divider	100K resistor
8	15K Ω 1/4W Resistor	Battery V/Divider	15K resistor
9	12VDC Adapter	12V 1A 5.5mm x 2.1mm	Power for Station
10	Plastic Container	48 oz, 7" x 10" x 2"	Station Enclosure - Zip-tied to block
11	Snap-in Support	PCB Mounting Post	Amazon PCB Support
12	Dupont Jumpers	Female to Male (10)	Component interconnects
13	Hot Glue		To secure items if required
	Network		
	SSID	URSchooI	
	Password	123456	
	IT Contact	Phone Number	email
	Jim Dandy	555-2134	jd@ITOffice.biz

Note:

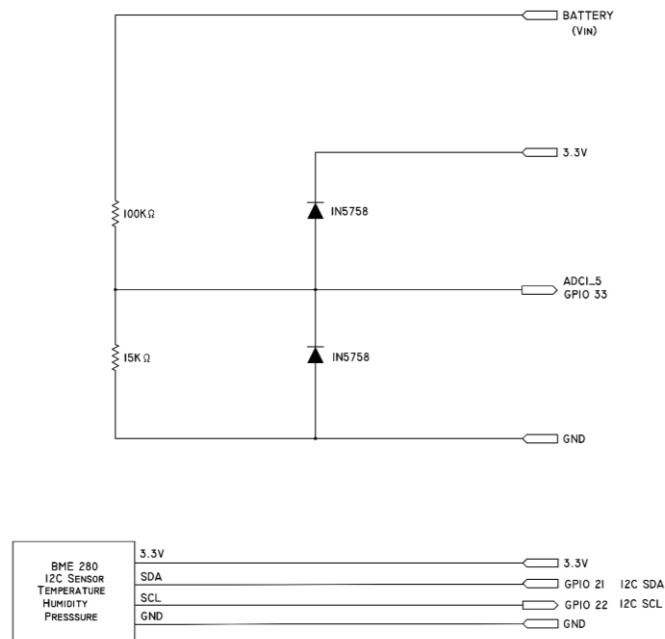
Resources listed are suggested components and vendors, you may use whatever source you see fit. It is recommended that component substitutes be selected based on similar specifications. Please consult your CIJE High School Mentor if in doubt prior to purchasing.

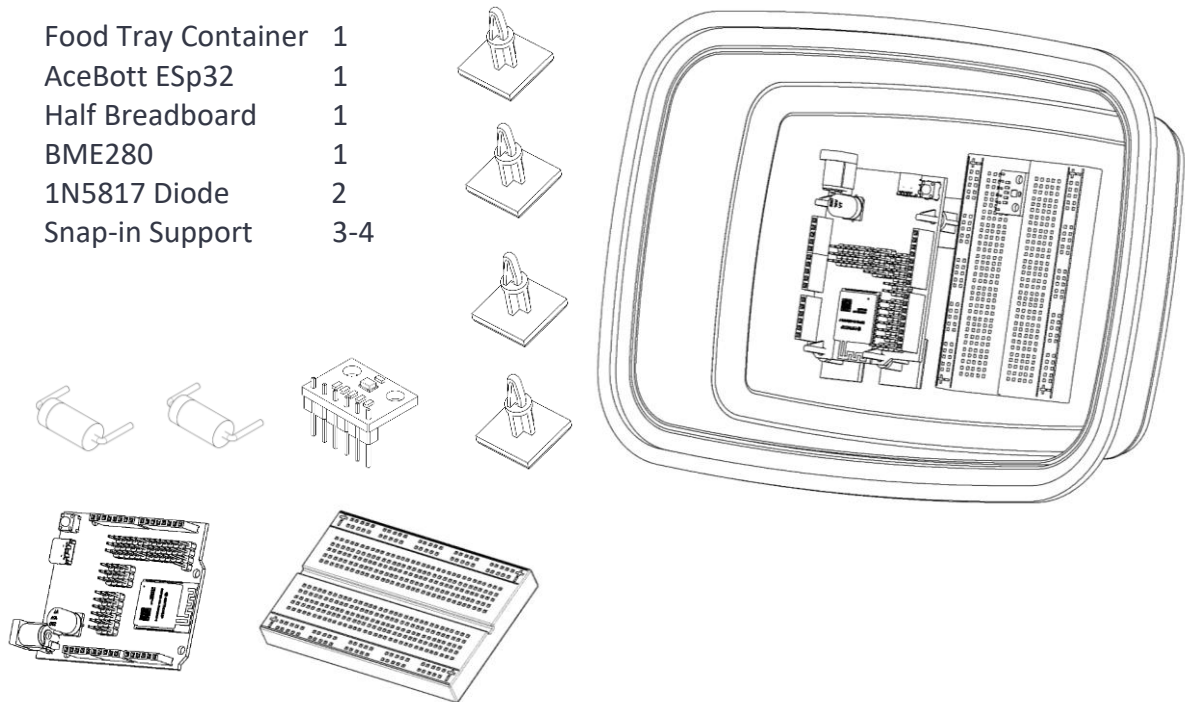
Weather Station Wiring for Anemometer, BME280 Sensor and Measuring Battery Voltage

CIJE WEATHER STATION ANEMOMETER WIRING



CIJE WEATHER STATION BME280





After gathering the above components, place them in the bottom of the Food Tray Container. Do a visual check first by placing them centered both vertically and horizontally next to each other. The easiest way to secure them is to use the adhesive Snap-in Support and the using the adhesive backing on the half breadboard. Mark or try positioning the ESP32 board and the half breadboard first and mark or identify where they go.

Steps:

- Insert the supports into the AceBott ESP32 board.
- Remove the adhesive, use the previous positioning and place it in the bottom, note it is suggested that once in place you apply hot glue to the adhesive pads to add another layer of attachment to the food tray.
- Remove the adhesive backing on the half breadboard and firmly place it next to the ESP32.
- You may insert the BME280 into the breadboard where you think it will work best.
 - The BME280 pins VCC, GND, SDA, SCL should be accessible via other pins on the breadboard.
 - You will connect pins via Dupont jumpers to the ESP32.

Now that you have mounted the components, make a couple of notes.

- The location of the programming port.
- The location of the power port.

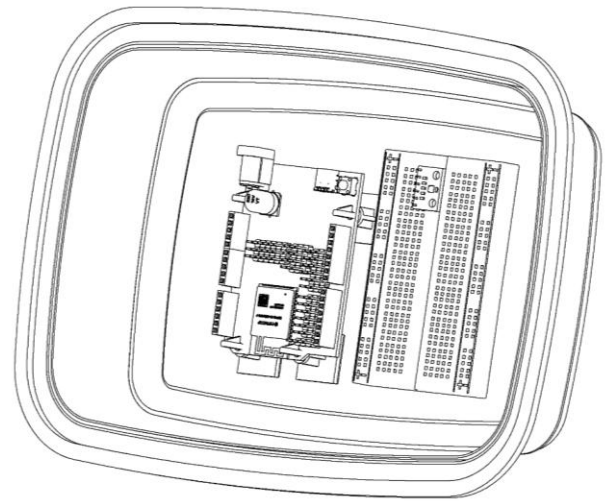
Using the above information consider how you want to connect those ports to the outside. The programming port will only be used initially to program the weather station and may only be required for changes. Power will be required to operate the weather station and should be both secure (not easily removed) and weatherproof.

We have designed a weather seal that you can 3D print if you like. We have a cutout template as well. A small USB C port plug is also included. You will find the 3D STL file and the cutout template in the weather station resource drive.

After using the cutout and inserting the weather seal you may want to use Silicone RTV to attach the seal. RTV is a good adhesive to use, it is weatherproof, remains pliable and is easy to remove.

You can use the container lid as the base to mount and seal the project. The weather station in its current configuration can be mounted in any orientation. However, if you plan on adding additional sensors it is recommended that the lid be mounted on the bottom and horizontal to the ground. You can secure the lid by any method that you prefer. We mounted the bottom using 4 holes and tie wraps.

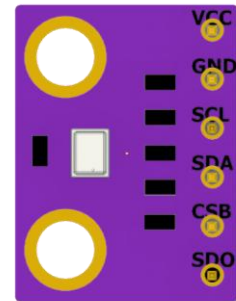
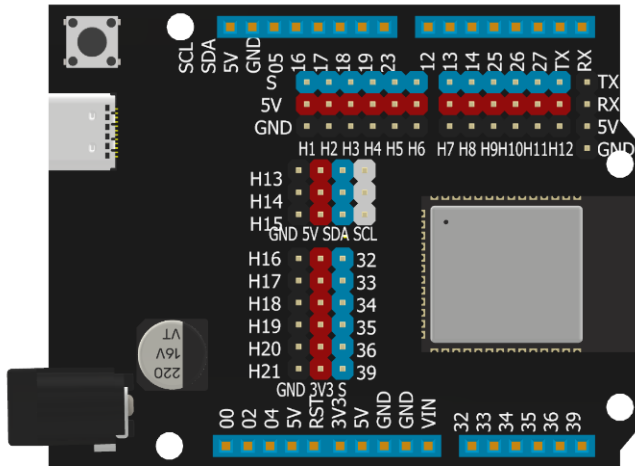
The bottom lid should have one or two small holes to allow for pressure equalization and moisture to exit. 1/16" holes are recommended. Larger holes may encourage insects to build nests in the enclosure and cause problems with the electronics.



Wiring:

Wiring the weather station will require minimal preparation.

- Acquire 4 Female to Male Dupont Jumpers.
- Using the following chart connect jumpers as shown in the wiring chart.



	Description	ESP32	ESP32 Pin	Sensor	Sensor Pin
1	DHT22 Environmental Sensor	5V	H1 5V	5V	VCC
		GND	H1 GND	GND	GND
		S	H1 5	D	D
2	100KΩ 1/4W Resistor	Receive the Battery voltage from the voltage divider network. Use pin 33 for the analog input.		Form a voltage divider network, with one end of 100KΩ going to the battery supply, and the other to the junction of the 15KΩ. The other end of the 15KΩ should go to GND.	
3	15KΩ 1/4W Resistor				
4	1N5817 Diode (2)	Overvoltage Protection	1N5817 Schottky Diode		1N5817 Diode (2)

Upon completion have someone else double check the connections.

Note:

Resources listed are suggested components and vendors, you may use whatever source you see fit. It is recommended that component substitutes be selected based on similar specifications. Please consult your CIJE High School Mentor if in doubt prior to purchasing.

Software:

This may be the most challenging part of this project. If you have never used an ESP32 before there are a few steps that we have to take before we can use the ESP32 with the Arduino IDE.

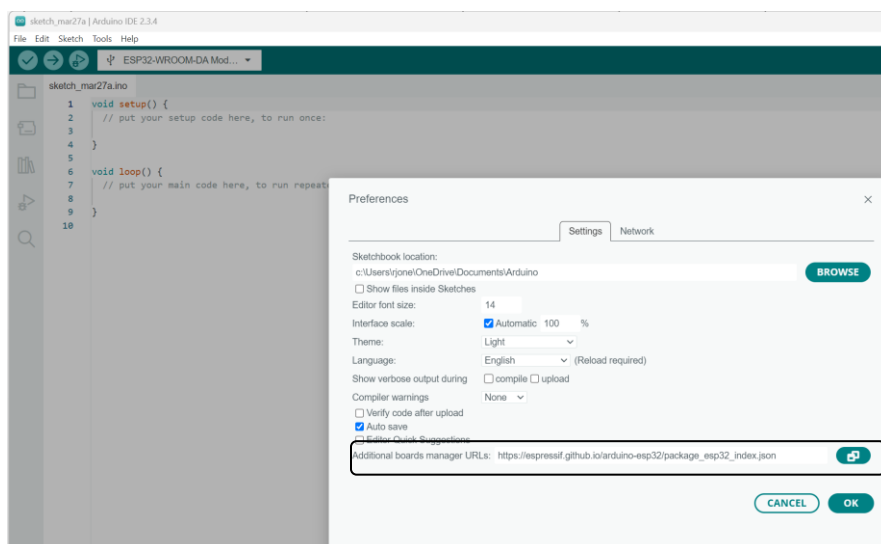
Assuming you have used the Arduino IDE before we will need to open the application and set some parameters:

Open the Arduino IDE and using the File Tab, scroll down to the dialog box called “Preferences.”

Notice in the dialog box the section a section that says:
Additional Boards
manager URLs.

Click on the green window selector on the right.

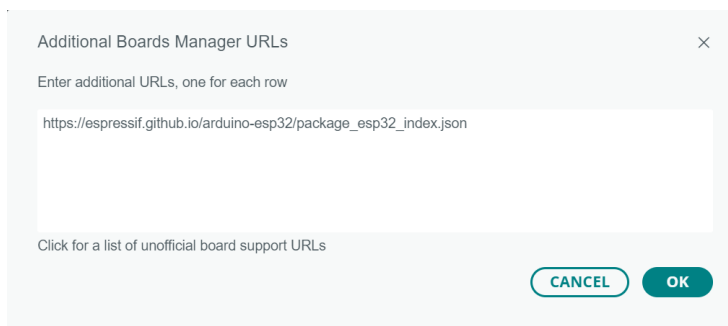
Here we will enter the URL that will point to descriptions of the Espressif.



https://espressif.github.io/arduino-esp32/package_esp32_index.json

Cut and paste the above URL into the URL dialog box. When complete “Click OK”.

This will begin a process that will request the Arduino IDE to go out on the Internet to download the various board descriptions.



Note:

- You must have internet access to download these files.
- Depending on your Internet access it may take up to a half hour or more to complete.
- You cannot proceed until this is accomplished.

Select the Board Manager to configure the IDE for ESP32 boards. On the Left Dialog tab, find and open the item marked as Board Manager (as shown on the right).

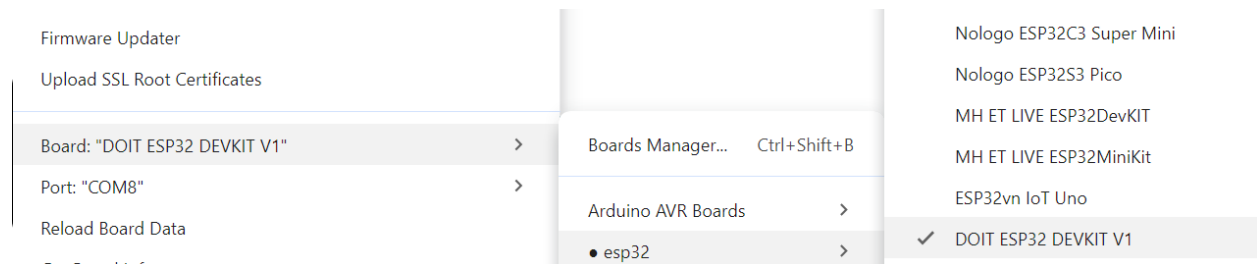
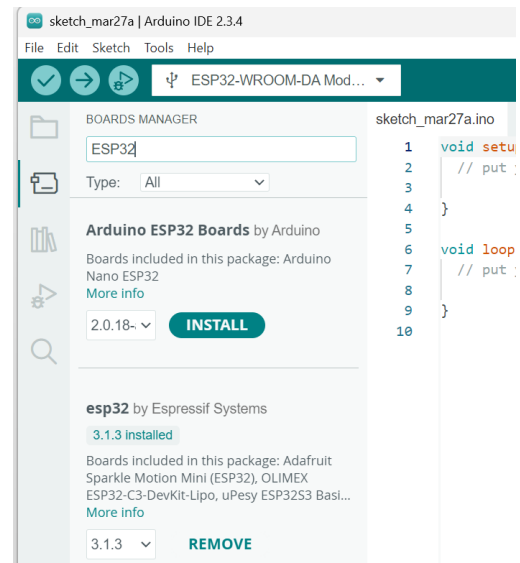
In the Boards Manager Search Box type “ESP32”. Typically, two suggestions will pop up.

- Arduino ESP32 Boards by Arduino
- esp32 by Espressif Systems

Install esp32 by Espressif Systems.

Installations will take a while so be patient.

Upon completion of the Board Manager move to the “Tools” tab and scroll down to Board and allow the dialog box to open on the right. Select esp32 and allow that dialog box to open. You will see possibly 50 different boards and selecting the “right one” is critical to get your code to compile.



Scroll through the esp32 board tab and look for:

- DOIT ESP32 DEVKIT V1

This module will populate the various parameters of the board into the IDE. If you plug in the ACEBOTT board now you will have to select the com port for programming.

Select from the Tools Com Drop Down Box.

Note:

Resources listed are suggested components and vendors, you may use whatever source you see fit. It is recommended that component substitutes be selected based on similar specifications. Please consult your CIJE High School Mentor if in doubt prior to purchasing.

The com port that the board is attached to.

In the example to the right
“COM8 Serial Port (USB)”
yours will probably be different.

Click “OK” when complete.

If you do not see an available COM port, please contact your instructor or IT specialist to identify the port location or Serial to USB driver for your PC (this process is different for PC’s, Mac’s and Chromebooks).

After completing these steps, we can begin configuring the IDE for the code, and sensors of your weather station.

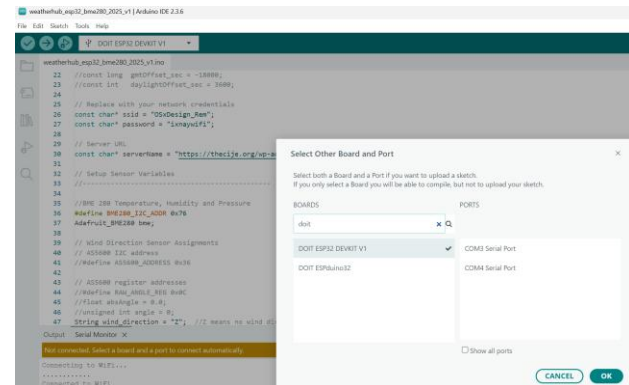
For the basic weather station using the BME280 and the ACEBOTT ESP32 go to the following URL and download the following Arduino Program Code:

Code Operation:

Lines 25-28 are code inclusion statements for the various libraries need to:

- Enable Wi-Fi operation.
- Enable Web Server communication.
- Provide driver interface for DHT Temperature and Humidity sensor.

```
26 #include <WiFi.h>
27 #include <HttpClient.h>
28 #include <DHT.h>
```



Lines 62-76 setup variables to use with:

- CIJE Weather Station ID # and Passkey, obtained by registering your station on the CIJE Weather Station Site
- Require Network credentials, SSID and Password
- URL location of the CIJE Weather station Server for data posting

```
60 // ===== NETWORK CONFIGURATION =====
61
62 // WiFi Configuration
63 const char* WIFI_SSID = "YOUR_SSID"; // Replace with your WiFi network name
64 const char* WIFI_PASSWORD = "YOUR_PASSWORD"; // Replace with your WiFi password
65 const int WIFI_TIMEOUT = 30000; // 30 seconds WiFi connection timeout
66 const int HTTP_TIMEOUT = 15000; // 15 seconds HTTP request timeout
67
68 // Weather Station Credentials
69 const int STATION_ID = #; // Replace with your registered station ID
70 const String PASSKEY = "XXXXX"; // Weather station passkey
71
72 // API Configuration
73 const String API_URL = "https://v0-cije-weather-hub.vercel.app/api/weather/submit";
74 const String USER_AGENT = "ESP32-WeatherStation-Test/1.0";
75 const String CONTENT_TYPE = "application/x-www-form-urlencoded";
76
```

Lines 30-60 contain variable assignments and default variable values:

- DHT22 sensor instance assignment and pin number
- Variables for
 - Wind Speed
 - Battery Voltage Sensor pin
 - Battery Voltage A/D count
 - Battery Voltage Analog value
 - Wi-Fi Connection Wait Timeout value

```

30 // ===== HARDWARE CONFIGURATION =====
31
32 // Pin Assignments
33 //DHT22 Temperature, Humidity
34 #define DHTPIN 5
35 #define DHTTYPE DHT22 // DHT 22 (AM2302), AM2321
36
37 DHT dht(DHTPIN, DHTTYPE);
38
39 #define ADC_PIN_WIND 34 // Analog pin for wind speed sensor (e.g., GPIO 36 is ADC1_CHANNEL_0)
40
41 // Hardware Specifications
42 const String BOARD_MODEL = "ESP32-WROOM-32";
43 const String SENSOR_MODEL_DHT22 = "DHT22";
44 const String SENSOR_MODEL_WIND = "Analog Anemometer";
45 const String PROJECT_NAME = "CIJE Weather Hub - ESP32 Station";
46 const String PROJECT_VERSION = "1.0.0";
47 const String STATION_NAME = "BJDelta";
48
49 // Sensor Specifications
50 const float DHT22_TEMP_MIN = -40.0; // BM280 minimum temperature (°C)
51 const float DHT22_TEMP_MAX = 80.0; // BM280 maximum temperature (°C)
52 const float DHT22_HUM_MIN = 0.0; // BM280 minimum humidity (%)
53 const float DHT22_HUM_MAX = 100.0; // BM280 maximum humidity (%)
54
55 // Wind Speed Calibration (Adjust these values based on your sensor's datasheet and calibration)
56 // Example: If 0-3.3V maps to 0-32.4 MPH, then 32.4 / 3.3 = 9.818
57 const float WIND_VOLTAGE_MAX = 3.3; // Max voltage output of your wind sensor
58 const float WIND_SPEED_MAX_MPH = 32.4; // Max wind speed in MPH corresponding to max voltage
59 const float WIND_CONVERSION = WIND_SPEED_MAX_MPH / WIND_VOLTAGE_MAX; // Conversion factor V -> MPH
60

```

Lines 104-141 provide initialization instructions in void setup()

- Serial Monitor (115200 Baud)
- DHT22 Initialization
- Find and verify DHT22 Sensor operation.
- Connect to Wi-Fi and verify operation.
- Post Weather Data to CIJE Weather Station Web Portal.

```

104 // ===== SETUP =====
105 void setup()
106 {
107     Serial.begin(115200);
108     delay(1000);
109
110     Serial.println("\n=====");
111     Serial.println(PROJECT_NAME);
112     Serial.println("=====");
113     Serial.println("DHT22 Data Pin: 2 GPIO ");
114     Serial.println("Wind Speed Analog Pin: GPIO " + String(ADC_PIN_WIND));
115     Serial.print("Sampling Interval: ");
116     Serial.print( SAMPLING_INTERVAL/60000.0);
117     Serial.println( " minutes");
118     Serial.println("=====");
119
120     // Initialize hardware
121     pinMode(STATUS_LED, OUTPUT);
122     digitalWrite(STATUS_LED, LOW);
123     float temperature, humidity, pressure;
124
125
126     // Initialize sensors
127     dht.begin();
128
129     // Initialize ADC for wind speed
130     analogReadResolution(12); // Set ADC resolution to 12 bits (0-4095)
131     analogSetAttenuation(ADC_11db); // Set attenuation to 11dB for 0-3.3V range
132     Serial.println("✅ ADC for wind speed initialized on GPIO " + String(ADC_PIN_WIND));
133
134     // Connect to WiFi
135     connectToWiFi();
136
137     Serial.println("Setup complete! Starting weather monitoring...");
138     Serial.println("=====");
139
140     //Read and Display Sensor Data
141     readAndSendWeatherData();
142 }

```

Lines 144-183 indicate null operation of the void loop().

- The primary function of the void loop() is to determine the post interval.
- Verify that the Wi-Fi connection is still active.
- LED Blink status light reporting.

```

144 // ===== MAIN LOOP =====
145 void loop()
146 {
147     unsigned long currentTime = millis();
148
149     // Check for serial commands
150     checkSerialCommands();
151
152     // Check if it's time for a sensor reading
153     if (currentTime - lastSensorReading >= SAMPLING_INTERVAL) {
154
155         // Ensure WiFi is connected
156         if (WiFi.status() != WL_CONNECTED) {
157             Serial.println("WiFi disconnected. Reconnecting...");
158             connectToWiFi();
159         }
160
161         if (WiFi.status() == WL_CONNECTED) {
162             readAndSendWeatherData();
163             lastSensorReading = currentTime;
164         }
165         else
166         {
167             Serial.println("✗ Cannot send data - WiFi not connected");
168             consecutiveFailures++;
169         }
170     }
171
172     // Handle consecutive failures
173     if (consecutiveFailures >= MAX_CONSECUTIVE_FAILURES)
174     {
175         Serial.println("⚠ Too many consecutive failures. Restarting ESP32...");
176         delay(RESTART_DELAY);
177         ESP.restart(); //Use ESP.abort() if reentry reset errors
178     }
179
180     // Blink status LED
181     blinkStatusLED();
182     delay(LOOP_DELAY);
183 }

```

Function Descriptions:

Lines 99-118 Provide the necessary code for connecting and authenticating with your Wi-Fi network.

- Using your SSID and Password begin the Wi-Fi connection process.
- Determine if the Wi-Fi connection has been established within the assigned time.
- Print the status of the attempted Wi-Fi communication.
- Upon exit of the function return a “True” or “False” status of the attempt

```

185 // ===== WIFI FUNCTIONS =====
186
187 void connectToWiFi()
188 {
189     Serial.println("Connecting to WiFi: " + String(WIFI_SSID));
190
191     WiFi.mode(WIFI_STA);
192     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
193
194     unsigned long startTime = millis();
195
196     while (WiFi.status() != WL_CONNECTED && (millis() - startTime) < WIFI_TIMEOUT) {
197         delay(500);
198         Serial.print(".");
199     }
200
201     if (WiFi.status() == WL_CONNECTED) {
202         Serial.println();
203         Serial.println("✅ WiFi connected successfully!");
204         Serial.println("IP Address: " + WiFi.localIP().toString());
205         Serial.println("Signal Strength: " + String(WiFi.RSSI()) + " dBm");
206         digitalWrite(STATUS_LED, HIGH);
207         consecutiveFailures = 0;
208     } else {
209         Serial.println();
210         Serial.println("❌ WiFi connection failed!");
211         Serial.println("Check SSID and password");
212         consecutiveFailures++;
213     }
214 }

```

Lines 216-227 is used to read the analog output of the wind generator and calculate the wind speed.

Note: This calculation may need to be adjusted for accurate readings.

```

216 // ===== SENSOR FUNCTIONS =====
217
218 float readWindSpeed()
219 {
220     int adcValue = analogRead(ADC_PIN_WIND);
221     // Convert ADC value (0-4095) to voltage (0-3.3V)
222     float voltage = (((float)adcValue / 4095.0) * 3.3) - 0.100; //Include ADC offset voltage 150mV
223     // Convert voltage to MPH using calibration factor
224     wind_speed = abs(voltage * WIND_CONVERSION);
225
226     return wind_speed;
227 }

```

Lines 229-268 Contains a set of instructions to read the weather sensors and to format the data for sending (posting).

- Verify DHT22 sensor working.
- Convert the DHT22 sensor data into a readable form.
- Notify the Blink status LED of sensor read.

```

229 void readAndSendWeatherData()
230 {
231     Serial.println("\n--- Reading Weather Data ---");
232
233     // Initialize sensors
234     // Reading temperature or humidity takes about 250 milliseconds!
235     // Sensor readings may also be up to 2 seconds 'old' (its a very slow sensor)
236     humidity = dht.readHumidity();
237     // Read temperature as Celsius (the default)
238     temperature = dht.readTemperature();
239     // Read temperature as Fahrenheit (isFahrenheit = true)
240     float f = dht.readTemperature(true);
241     // Compute heat index in Fahrenheit (the default)
242     float hif = dht.computeHeatIndex(f, humidity);
243     // Compute heat index in Celsius (isFahrenheit = false)
244     float hic = dht.computeHeatIndex(temperature, humidity, false);
245
246     readWindSpeed();
247
248     Serial.println("Temperature: " + String(temperature, 2) + "°C");
249     Serial.println("Humidity: " + String(humidity, 2) + "%");
250     Serial.println("Wind Speed: " + String(wind_speed, 2) + " mph");
251
252     blinkSuccess();
253
254     // Send data to server
255     bool success = sendWeatherData(temperature, humidity, wind_speed);
256
257     if (success)
258     {
259         Serial.println("✅ Data sent successfully");
260         consecutiveFailures = 0;
261         blinkSuccess();
262     } else {
263         Serial.println("❌ Failed to send data");
264         consecutiveFailures++;
265     }
266
267     Serial.println("--- End Reading ---\n");
268 }

```

Lines 279-336 Contains a lengthy set of instructions to perform posting weather data to the CIJE Weather Station Portal.

Lines 280-284.

- Verify Wi-Fi is operational before Posting Data, if not post “Wi-Fi Disconnected”.

Lines 286-290 Form the post header.

Lines 293-304 Collect and post the sensor data.

Lines 306-335 Verify and post diagnostic data regarding the data and posting process.

Lines 418-423 Contain code for reading and calculating battery voltage.

```

418 void readBatteryVoltage()
419 {
420     // Calculate battery voltage
421     battDigital = analogRead(battVoltagePin);
422     battery = (7.09 * ((3.30 / 4095.0) * (battDigital - wSpdOffset))); //
423 }

```

The previous code description outlines a basic Weather Station. Additional sensors and operations can be added to advance the capability of the Weather Station for Intermediate, and Advanced Operation.